UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO

ERASMO DA SILVA SÁ JUNIOR
JOSÉ RIQUELME TEIXEIRA DA SILVA
VICTOR ANDRÉ LOPES BRASILEIRO

SALA DE AULA 3D NAVEGÁVEL EM OPENGL: relatório técnico do projeto da AB2 de Computação Gráfica

MACEIÓ–AL
2026

ERASMO DA SILVA SÁ JUNIOR
JOSÉ RIQUELME TEIXEIRA DA SILVA
VICTOR ANDRÉ LOPES BRASILEIRO

SALA DE AULA 3D NAVEGÁVEL EM OPENGL: relatório técnico do projeto da AB2 de Computação Gráfica

Relatório técnico apresentado à disciplina de Computação Gráfica do Instituto de Computação da Universidade Federal de Alagoas, como requisito parcial para a avaliação da AB2.

Professor: Marcelo Costa Oliveira

Maceió–AL
2026

RESUMO

Este relatório descreve o projeto da AB2 de Computação Gráfica: uma sala de aula tridimensional navegável, escrita em C++17 com OpenGL 3.3 no perfil core, GLFW, GLAD, GLM e stb_image. A sala de 8 × 3 × 7 m reúne carteiras, cadeiras, quadro, mesa do professor, luminárias com interruptor, ventilador de parede, projetor, tela retrátil e lixeira, todos modelados no código a partir de primitivas e organizados num grafo de cena hierárquico. A cena é iluminada pelo modelo de Blinn-Phong por fragmento, com luzes pontuais atenuadas, luz direcional de preenchimento, materiais por peça e texturas. O usuário navega com câmera livre (teclado e mouse), inspeciona cada objeto isoladamente e interage com a sala: liga e desliga as luzes, abre a tela retrátil, liga o ventilador e o projetor e troca os slides. As três transformações pedidas na avaliação são detalhadas com suas matrizes: a escala em Y com pivô no topo da lona da tela retrátil, a rotação das pás do ventilador em torno do eixo do motor, integrada no tempo, e a projeção do slide por textura projetiva, em que o projetor é tratado como uma segunda câmera. O projeto compila com CMake e g++ no Windows (MinGW-w64) e no Linux, com as bibliotecas embutidas no repositório, e o build limpo foi verificado em três sistemas.

Palavras-chave: computação gráfica; OpenGL; transformações geométricas; iluminação; textura projetiva.

ABSTRACT

This report describes the second assessment (AB2) project of the Computer Graphics course: a navigable 3D classroom written in C++17 with core-profile OpenGL 3.3, GLFW, GLAD, GLM and stb_image. The 8 × 3 × 7 m room contains student desks, chairs, a whiteboard, a teacher's desk, ceiling lamps with a wall switch, a wall fan, a projector, a retractable screen and a trash can, all modeled in code from primitives and arranged in a hierarchical scene graph. The scene is lit with per-fragment Blinn-Phong shading, attenuated point lights, a directional fill light, per-part materials and textures. The user moves a free camera with keyboard and mouse, inspects each object in isolation and interacts with the room: switching the lights, opening the screen, turning on the fan and the projector and changing slides. The three transformations required by the assessment are detailed with their matrices: a Y scale pivoted at the top of the screen canvas, the rotation of the fan blades around the motor axis integrated over time, and the slide projection through projective texture mapping, which treats the projector as a second camera. The project builds with CMake and g++ on Windows (MinGW-w64) and Linux with bundled libraries, and a clean build was verified on three systems.
Keywords: computer graphics; OpenGL; geometric transformations; lighting; projective texture mapping.

LISTA DE ILUSTRAÇÕES

Figura 1 – Vista inicial da sala, com as luzes acesas 8
Figura 2 – Camadas do código e sentido das dependências 11
Figura 3 – Objetos no modo de inspeção 14
Figura 4 – Projeção do slide na tela retrátil 19
Figura 5 – Tela retrátil aberta, com o projetor desligado 23

LISTA DE QUADROS

Quadro 1 – Bibliotecas usadas 10
Quadro 2 – Objetos da sala 13
Quadro 3 – Sistemas testados 24
Quadro 4 – Responsabilidades por integrante 25
Quadro 5 – Comandos do programa 28

LISTA DE CÓDIGOS

3.1 **Desenho recursivo do grafo de cena (`src/gfx/scene_node.cpp`, sem comentários)** 12
5.1 **Escala da lona (`src/objects/projection_screen.cpp`, sem comentários)** 15
5.2 **Rotação do rotor (`src/objects/fan.cpp`, sem comentários)** 16
5.3 **Matrizes do projetor (`src/scene/classroom.cpp`, sem comentários)** . . . 18
5.4 **Luz do projetor no fragmento (`shaders/basic.frag`, sem comentários)** . 18
9.1 **Compilação no Windows (PowerShell)** 24
9.2 **Compilação no Linux** 24

SUMÁRIO

1	**INTRODUÇÃO**	**8**
2	**TECNOLOGIAS**	**10**
3	**ARQUITETURA**	**11**
3.1	**Grafo de cena**	**11**
3.2	**Loop principal**	**12**
4	**MODELAGEM E MONTAGEM DA SALA**	**13**
5	**TRANSFORMAÇÕES GEOMÉTRICAS**	**15**
5.1	**Escala: tela retrátil**	**15**
5.2	**Rotação: ventilador**	**16**
5.3	**Projeção: slide do projetor**	**17**
6	**ILUMINAÇÃO, MATERIAIS E TEXTURAS**	**20**
6.1	**Modelo de Blinn-Phong**	**20**
6.2	**Matriz das normais**	**20**
6.3	**Materiais e texturas**	**20**
7	**CÂMERA E INTERAÇÃO**	**22**
8	**RESULTADOS**	**23**
9	**COMPILAÇÃO E EXECUÇÃO**	**24**
10	**DIVISÃO DE TAREFAS**	**25**
11	**CONCLUSÃO**	**26**
	Referências	**27**
	APÊNDICE A – LISTA DE COMANDOS	**28**

1 INTRODUÇÃO

O objetivo do projeto é construir, com OpenGL moderno, uma sala de aula tridimensional que possa ser percorrida e manipulada em tempo real. A avaliação pede objetos modelados pela equipe, iluminação, texturas, câmera navegável e interações que exercitem transformações geométricas: uma escala, uma rotação e uma projeção, cada uma ligada a um objeto da sala.

O resultado é um único executável que abre a janela mostrada na Figura 1. A sala tem 6 carteiras e 6 cadeiras, quadro branco, mesa do professor, quatro luminárias de teto com interruptor de parede, ventilador de parede, projetor preso ao teto, tela retrátil e lixeira. Nenhum modelo foi importado: toda a geometria é gerada no código, a partir de caixas, cilindros, planos e malhas montadas vértice a vértice.

Figura 1 – Vista inicial da sala, com as luzes acesas



Fonte: Elaborada pelos autores (2026).

As interações pedidas foram distribuídas assim: a tela retrátil usa a **escala** para abrir e recolher a lona; o ventilador usa a **rotação** para girar as pás; e o projetor usa a **projeção** para lançar o slide sobre a lona, o quadro ou a parede. Além delas, a sala tem luzes que acendem e apagam pelo interruptor.

O Capítulo 2 apresenta as tecnologias e o Capítulo 3, a organização do código. O Capítulo 4 descreve a modelagem e a montagem da sala, e o Capítulo 5 detalha as três transformações. O Capítulo 6 trata da iluminação, dos materiais e das texturas, e o Capítulo 7, da câmera e da interação. Os resultados estão no Capítulo 8; a compilação, no Capítulo 9; a divisão de tarefas, no

Capítulo 10. O Capítulo 11 conclui o relatório. A lista completa de comandos está no Apêndice A. O texto segue o manual de normalização da UFAL (Vale; Lenzi, 2025).

2 TECNOLOGIAS

O programa é escrito em C++17 e usa o OpenGL 3.3 no perfil *core* (Segal; Akeley, 2010), que exige o pipeline programável: não há `glBegin/glEnd` nem pilha de matrizes do pipeline fixo. Toda geometria vai para a GPU em *buffers* (VAO, VBO e EBO) e é desenhada por um par de shaders GLSL (`shaders/basic.vert` e `shaders/basic.frag`). As matrizes são calculadas na CPU e enviadas como *uniforms*.

As bibliotecas de terceiros ficam no próprio repositório, em `lib/`, e são compiladas junto com o projeto pelo CMake (Quadro 1). Assim, o build não depende de pacotes instalados no sistema, o que simplifica a compilação em máquinas diferentes.

Quadro 1 – Bibliotecas usadas

Biblioteca	**Uso no projeto**
GLFW 3.5.1 (GLFW, 2026)	Janela, contexto OpenGL, teclado, mouse e foco
GLAD (Herberth, 2026)	Carrega as funções do OpenGL 3.3 core fornecidas pelo driver
GLM 1.0.3 (G-Truc Creation, 2026)	Vetores, matrizes, `lookAt`, `perspective`, `rotate`, `scale`
stb_image (Barrett, 2026)	Leitura das texturas PNG

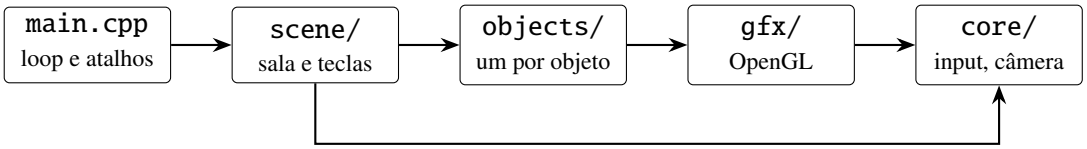
Fonte: Elaborado pelos autores (2026).

O build usa CMake com *presets* (`CMakePresets.json`): `windows`, com o g++ do MinGW-w64 (MSYS2), e `linux`, com o g++ do sistema e os headers do X11. O Capítulo 9 traz os passos.

3 ARQUITETURA

O código é dividido em quatro camadas, com headers em `include/` e implementações em `src/`. Cada camada só depende das que estão à sua direita na Figura 2.

Figura 2 – Camadas do código e sentido das dependências



Fonte: Elaborada pelos autores (2026).

- `core/`: entrada (`Input`, sobre os *callbacks* do GLFW) e câmera, sem OpenGL;
- `gfx/`: malhas, primitivas, shader, textura, material, luzes, nó de cena e o *renderer*. Desenha qualquer árvore de nós e não conhece a sala;
- `objects/`: um módulo por objeto. O objeto é modelado no próprio espaço, com o pivô na origem, e expõe métodos como `toggle()` e `update(dt)`. Ele não lê o teclado nem sabe onde está na sala;
- `scene/`: a classe `Classroom` posiciona os objetos, monta as luzes, define os alvos de inspeção e liga cada tecla ao método do objeto correspondente.

Essa divisão deixa a tabela de atalhos num lugar só e permitiu que cada integrante trabalhasse nos próprios arquivos, com conflitos de *merge* restritos às poucas linhas que posicionam os objetos na sala.

3.1 Grafo de cena

A cena é uma árvore de `SceneNode`. Cada nó tem uma malha opcional, uma matriz `transform` relativa ao pai, um material opcional e um indicador de visibilidade. Ao desenhar, a matriz de modelo do nó é o produto da matriz do pai pela do próprio nó:

$$M_{\text{nó}} = M_{\text{pai}} \cdot T_{\text{nó}} \tag{3.1}$$

e os filhos recebem $M_{\text{nó}}$ como matriz do pai (Código 3.1). Mudar a `transform` de uma articulação move tudo o que está pendurado nela, sem tocar nos vértices. As três transformações do Capítulo 5 usam exatamente isso: cada uma altera só a matriz de um nó articulado. O material segue a mesma lógica: um nó sem material herda o do pai.

Código 3.1 – Desenho recursivo do grafo de cena (`src/gfx/scene_node.cpp`, sem comentários)

```
void SceneNode::draw(const Shader& shader, const Material& inheritedMaterial,
                     const glm::mat4& parentModel) const {
    if (!visible) {
        return;
    }
    const glm::mat4 model = parentModel * transform;
    const Material& currentMaterial = material != nullptr ? *material :
    inheritedMaterial;
    if (mesh != nullptr) {
        shader.setMat4("uModel", model);
        shader.setMat3("uNormalMatrix", glm::transpose(glm::inverse(glm::mat3(model)
    )));
        currentMaterial.apply(shader);
        mesh->draw();
    }
    for (const auto& child : children_) {
        child->draw(shader, currentMaterial, model);
    }
}
```

Fonte: Elaborado pelos autores (2026).

3.2 Loop principal

A cada quadro, `main.cpp` lê os eventos, calcula o intervalo de tempo Δt em segundos, atualiza a sala (interações e animações), trata a inspeção, move a câmera e desenha a árvore. O Δt é limitado a $[0;\ 0{,}1]$ s: depois de uma pausa longa (janela arrastada, perda de foco), o excesso é descartado em vez de fazer as animações saltarem. Todas as animações usam Δt, então a velocidade é a mesma em qualquer taxa de quadros.

4 MODELAGEM E MONTAGEM DA SALA

Os objetos foram modelados em metros, com Y para cima, e com a frente de cada objeto voltada para +Z no espaço local. As primitivas (`gfx/primitives`) geram caixas, cilindros e planos com normais e coordenadas de textura em metros; peças que nenhuma primitiva produz, como a parede oca da lixeira ou o refletor em "W" da luminária, são montadas vértice a vértice no módulo do objeto. O Quadro 2 resume os objetos.

Quadro 2 – Objetos da sala

Objeto (classe)	**Modelagem**	**Qtd.**
Sala (`Room`)	Piso, paredes e teto com espessura, rodapés; interior de 8 × 3 × 7 m	1
Carteira (`StudentDesk`)	Tampo de 0,60 × 0,45 m a 0,75 m, tubos de 22 mm, porta-livros de arame	6
Cadeira (`Chair`)	Assento de 0,40 × 0,40 m, encosto inclinado 8°, pernas tubulares	6
Quadro (`Board`)	Moldura de 3,00 × 1,20 m, bandeja, apagador e dois marcadores	1
Mesa do professor (`TeacherDesk`)	Tampo de 1,20 × 0,60 m, painéis de MDF, gaveteiro com puxadores	1
Luminária (`Lamp`)	Calha de 1,27 × 0,25 m, dois tubos T8, refletor em "W" e aletas	4
Interruptor (`LightSwitch`)	Espelho de 0,08 × 0,12 m e tecla basculante	1
Ventilador (`Fan`)	Suporte, motor, grades e rotor com três pás de ~0,50 m	1
Projetor (`Projector`)	Placa, haste, corpo de 0,30 × 0,10 × 0,25 m e lente separada	1
Tela retrátil (`ProjectionScreen`)	Caixa de 2,10 m, lona de 2,0 × 1,5 m, barra e puxador	1
Lixeira (`TrashCan`)	Tronco de cone oco, 0,30 m de boca e 0,40 m de altura, parede de 6 mm	1

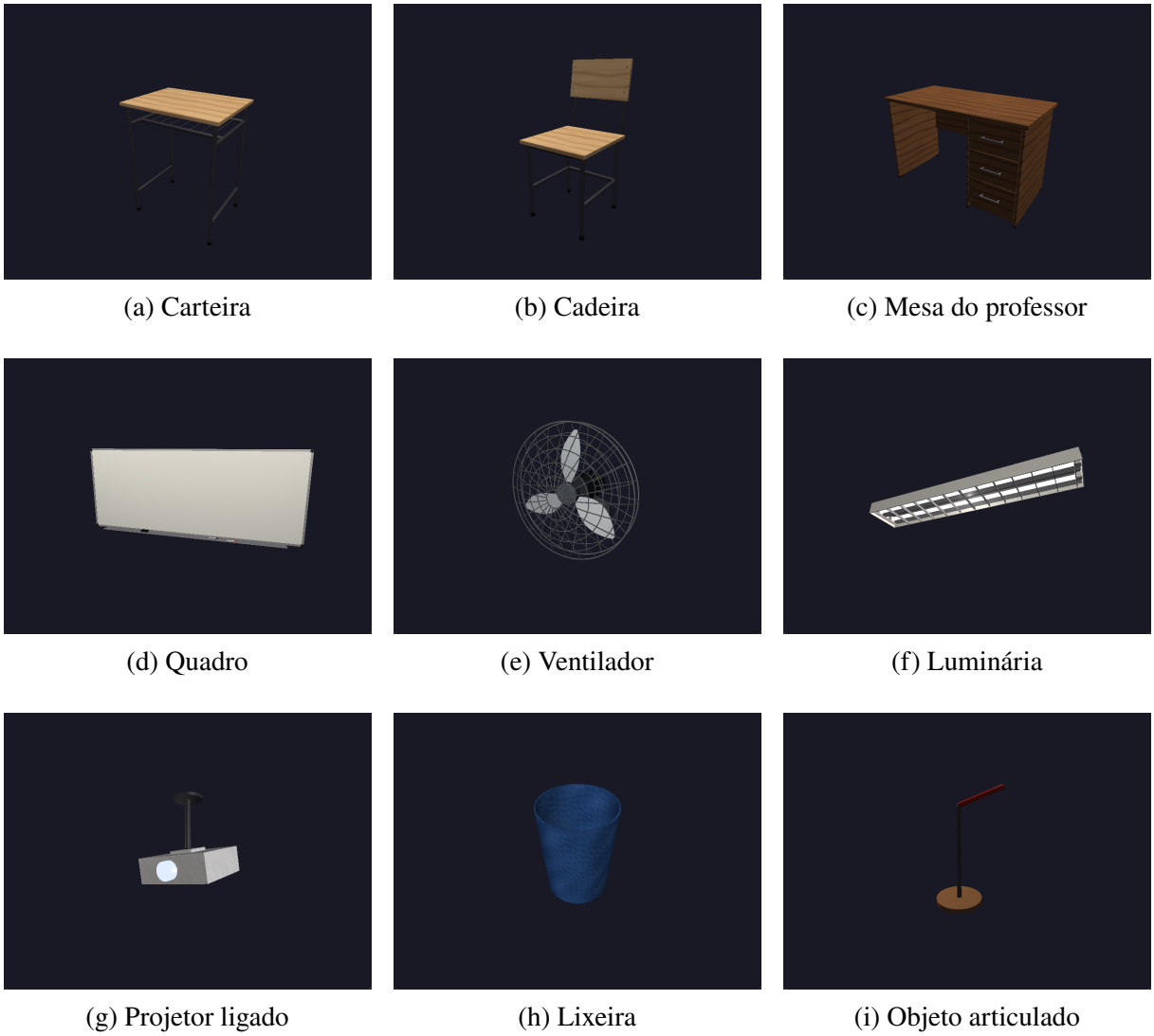
Fonte: Elaborado pelos autores (2026).

Cada objeto tem um método `addTo(pai, placement)`, que pendura uma raiz nova na árvore com a matriz de posicionamento recebida. As malhas são criadas uma vez e compartilhadas: as seis carteiras, por exemplo, usam os mesmos buffers e materiais, com nós independentes. A `Classroom` faz a montagem: as carteiras ficam em duas colunas (X = ±1,25 m) e três fileiras, com corredor central de 1,90 m; cada cadeira fica 0,55 m atrás da carteira e girada 180° em Y, olhando para o quadro; a tela fica no alto da parede frontal, à frente do quadro; o projetor fica no teto, a cerca de 2,9 m da lona.

O alinhamento entre projetor e tela é calculado, e não ajustado à mão. O centro da lona é levado do espaço da tela para o mundo e, depois, do mundo para o espaço do projetor, pela inversa do posicionamento dele; `Projector::aimAt` inclina a cabeça para o eixo da lente passar por esse ponto (cerca de 12,5°). Mover a tela ou o projetor realinha os dois sozinho.

A Figura 3 mostra os objetos no modo de inspeção, que esconde a sala e as outras instâncias e enquadra a câmera num exemplar.

Figura 3 – Objetos no modo de inspeção



(a) Carteira (b) Cadeira (c) Mesa do professor

(d) Quadro (e) Ventilador (f) Luminária

(g) Projetor ligado (h) Lixeira (i) Objeto articulado

Fonte: Elaborada pelos autores (2026).

5 TRANSFORMAÇÕES GEOMÉTRICAS

As três transformações usam matrizes 4 × 4 em coordenadas homogêneas (Shirley; Marschner, 2016). Um ponto $\mathbf{p} = (x, y, z, 1)^T$ é transformado por $\mathbf{p}' = M\mathbf{p}$ e, num produto $M = A \cdot B$, a matriz da direita é aplicada primeiro. As matrizes de translação, escala e rotação em torno de Z são:

$$T(\mathbf{t}) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{5.1}$$

Escala e rotação acontecem em torno da origem. Para escalar ou girar em torno de um pivô $\mathbf{c}$, o objeto é modelado com o pivô na origem do nó e a matriz do nó leva essa origem até $\mathbf{c}$: $T(\mathbf{c}) \cdot X$, com X sendo a escala ou a rotação. Os trechos de código citados a seguir estão marcados no repositório com o comentário `AB2 ·` e podem ser listados com `git grep -n "AB2 ·"`.

5.1 Escala: tela retrátil

A tela tem uma caixa no alto da parede e uma lona de 2,0 × 1,5 m que desce por uma fenda. A lona é filha de uma articulação cuja origem é o topo da própria lona: a malha vai de $y = 0$ (topo) a $y = -1{,}5$ m (base). Para abrir e recolher, só a matriz dessa articulação muda:

$$M_{\text{lona}} = T(\mathbf{p}_{\text{fenda}}) \cdot S(1,\ s,\ 1), \qquad s \in [0, 1], \tag{5.2}$$

em que $s = 0$ é a tela recolhida e $s = 1$, aberta. Lida da direita para a esquerda, a escala em Y acontece em torno do topo da lona, que é a origem do nó; só depois a translação leva esse topo até a fenda. Um vértice de altura local y vai para $y_{\text{fenda}} + s \cdot y$: o topo ($y = 0$) não se move e a base sobe ($-1{,}5s$). A lona encolhe para cima sem descolar da caixa, como uma tela real enrolando no rolo (Código 5.1). A barra inferior não é escalada, só transladada para acompanhar a base: $T(\mathbf{p}_{\text{fenda}} + (0, -1{,}5s, 0))$.

Código 5.1 – Escala da lona (`src/objects/projection_screen.cpp`, sem comentários)

```
void ProjectionScreen::applyOpenAmount() {
    canvasMaterial_.uvScale.y = kCanvasHeight * openAmount_ / kCanvasTexturePeriod;
    if (root_ == nullptr) {
        return;
    }
    canvasPivot_->transform = glm::scale(translated(kSlot), {1.0f, openAmount_, 1.0f
    });
    canvasPivot_->visible = openAmount_ > kMinVisibleAmount;
    bottom_->transform = translated(kSlot + glm::vec3(0.0f, -kCanvasHeight *
    openAmount_, 0.0f));
```

```
}
```

Fonte: Elaborado pelos autores (2026).

A tecla `T` inverte o sentido do movimento. Um progresso $t \in [0, 1]$ avança linearmente com o tempo (curso completo em 2,5 s) e a escala é suavizada por $s = 3t^2 - 2t^3$ (*smoothstep*), que parte e chega com velocidade nula. Inverter no meio do curso é contínuo, porque só o sentido de t muda. Dois detalhes decorrem da escala não uniforme:

- **Textura**: como a escala encurta a lona mas as coordenadas de textura continuam de 0 a 1, a trama do tecido achataria em Y. A repetição vertical é ajustada para $1{,}5s/0{,}25$, mantendo o período de 25 cm;
- **Normais**: a matriz das normais é a inversa transposta (seção 6.2); com $s = 0$ ela não existe (matriz singular), por isso a lona fica invisível abaixo de $s = 0{,}001$.

5.2 Rotação: ventilador

O ventilador tem um nó fixo (suporte, motor, eixo e grades) e um nó *rotor* com as três pás e a tampa central. As pás são modeladas em volta da origem do rotor, que coincide com o eixo do motor (eixo Z local). A matriz do rotor é

$$M_{\text{rotor}} = T(\mathbf{c}_{\text{rotor}}) \cdot R_z(\theta), \tag{5.3}$$

ou seja, gira as pás em torno do eixo e depois as leva ao centro do motor (Código 5.2). Os vértices nunca são recalculados: a cada quadro só muda θ. Grades e motor, no nó fixo, não giram.

O ângulo é integrado no tempo pelo método de Euler explícito. Com a velocidade angular ω (graus por segundo) e o alvo ω^* (0 desligado, ou 720, 1080 ou 1440 °/s, isto é, 2, 3 ou 4 voltas/s):

$$\omega_{k+1} = \begin{cases} \min(\omega_k + a_\uparrow \, \Delta t, \ \omega^*) & \text{se } \omega_k < \omega^* \\ \max(\omega_k - a_\downarrow \, \Delta t, \ \omega^*) & \text{caso contrário} \end{cases} \tag{5.4}$$

$$\theta_{k+1} = \theta_k + \omega_{k+1} \, \Delta t, \tag{5.5}$$

com $a_\uparrow = 960$ °/s² (até 1,5 s para atingir a velocidade máxima) e $a_\downarrow = 480$ °/s² (até 3 s para parar). Como tudo é multiplicado por Δt, o giro por segundo é o mesmo a 30 ou a 144 quadros por segundo. O ângulo é reduzido a $[-180°, 180°]$ a cada passo para não perder precisão no `float` depois de horas ligado.

Código 5.2 – Rotação do rotor (`src/objects/fan.cpp`, sem comentários)

```
void Fan::setRotorAngle(float degrees) {
    if (!std::isfinite(degrees)) {
        return;
    }
    rotorAngle_ = std::remainder(degrees, 360.0f);
    if (rotor_ != nullptr) {
```

```
        rotor_->transform = glm::rotate(translated(kRotorCenter), glm::radians(
    rotorAngle_), {0, 0, 1});
    }
}

void Fan::update(float deltaTime) {
    const float target = on_ ? kSpeeds[speedLevel_] : 0.0f;
    if (speed_ < target) {
        speed_ = std::min(speed_ + kSpinUp * deltaTime, target);
    } else {
        speed_ = std::max(speed_ - kSpinDown * deltaTime, target);
    }
    if (speed_ > 0.0f) {
        setRotorAngle(rotorAngle_ + speed_ * deltaTime);
    }
}
```

Fonte: Elaborado pelos autores (2026).

As velocidades ficam bem abaixo das de um ventilador real (cerca de 20 voltas/s) por causa da amostragem temporal. Com três pás, o rotor é simétrico a cada 120°; se o giro entre dois quadros passar de 60°, o olho associa cada pá à pá vizinha e o rotor parece girar devagar, parado ou para trás (efeito "roda de carroça"). A 4 voltas/s e 60 quadros por segundo, o passo é de 24° por quadro; mesmo a 30 quadros por segundo, de 48°. A 20 voltas/s seriam 120° por quadro, e as pás pareceriam paradas.

5.3 Projeção: slide do projetor

O slide não é uma textura colada na lona. O projetor é tratado como uma luz que carrega uma imagem, pela técnica de textura projetiva (Segal; Korobkin *et al.*, 1992): cada fragmento da cena descobre em que ponto da imagem do projetor ele está e recebe a luz daquele texel. A imagem cai onde o feixe bater: na lona aberta, ou no quadro e na parede com a tela recolhida.

A pergunta "onde este ponto cai na imagem?" é a mesma que a câmera responde ao desenhar a cena. Por isso o projetor ganha uma matriz de visualização e uma de projeção próprias, como uma câmera posta na lente:

$$V_{\text{proj}} = \text{lookAt}(\mathbf{l},\ \mathbf{l}+\mathbf{d},\ \mathbf{u}), \tag{5.6}$$

$$P_{\text{proj}} = \text{perspective}(\mathit{fov}_y,\ 4/3,\ 0{,}1\ \text{m},\ 10\ \text{m}), \tag{5.7}$$

em que $\mathbf{l}$, $\mathbf{d}$ e $\mathbf{u}$ são o centro, o eixo e o "cima" da lente levados ao mundo. O produto $P_{\text{proj}}V_{\text{proj}}$ é calculado uma vez, na CPU (Código 5.3), e enviado ao fragment shader, onde cada fragmento de posição $\mathbf{p}$ percorre a cadeia

$$\mathbf{c} = P_{\text{proj}}V_{\text{proj}}\begin{bmatrix}\mathbf{p}\\ 1\end{bmatrix}, \qquad \mathbf{n} = \frac{(c_x, c_y, c_z)}{c_w} \in [-1,1]^3, \qquad (u,v) = 0{,}5\,(n_x, n_y) + 0{,}5 \in [0,1]^2. \tag{5.8}$$

A divisão por c_w é a divisão de perspectiva que a GPU faz sozinha com `gl_Position`; aqui ela é feita à mão. Fragmentos com $c_w \le 0$ (atrás da lente) ou com $\mathbf{n}$ fora do cubo $[-1,1]^3$ (fora da imagem, antes do plano próximo ou depois do distante) não recebem luz (Código 5.4).

O ângulo de abertura vem da geometria real. A lente fica a $d = 2{,}89$ m do centro da lona e a imagem ocupa $w = 1{,}84$ m de largura (92% dos 2,0 m da lona):

$$fov_x = 2\arctan\frac{w/2}{d} = 2\arctan\frac{0{,}92}{2{,}89} \approx 35{,}3^\circ, \qquad fov_y = 2\arctan\frac{\tan(fov_x/2)}{4/3} \approx 26{,}9^\circ. \quad (5.9)$$

Como o projetor está acima do centro da lona e inclinado para baixo, a borda inferior da imagem fica mais longe da lente (cerca de 3,05 m) que a superior (2,73 m) e sai mais larga: a imagem forma um trapézio (*keystone*), como num projetor real sem correção. A margem de 8% evita que a base passe da lona.

Código 5.3 – Matrizes do projetor (`src/scene/classroom.cpp`, sem comentários)

```
const glm::vec3 lensPosition = toWorld(projectorPlacement, projector_.lensCenter());
const glm::mat3 projectorRotation(projectorPlacement);
const glm::vec3 lensDirection = glm::normalize(projectorRotation * projector_.
    lensDirection());
const glm::vec3 lensUp = glm::normalize(projectorRotation * projector_.lensUp());
const float throwDistance = glm::distance(lensPosition, screenCenter);
const float halfImageWidth = kImageFill * screen_.canvasSize().x * 0.5f;
const float fovX = 2.0f * std::atan(halfImageWidth / throwDistance);
const float fovY = 2.0f * std::atan(std::tan(fovX * 0.5f) / kSlideAspect);
const glm::mat4 projectorView = glm::lookAt(lensPosition, lensPosition +
    lensDirection, lensUp);
const glm::mat4 projectorProjection = glm::perspective(fovY, kSlideAspect,
    kProjectorNear, kProjectorFar);
lighting_.projector.viewProjection = projectorProjection * projectorView;
```

Fonte: Elaborado pelos autores (2026).

Código 5.4 – Luz do projetor no fragmento (`shaders/basic.frag`, sem comentários)

```
vec3 projectorLight(vec3 N, vec3 albedo) {
    if (!uProjector.enabled) {
        return vec3(0.0);
    }
    vec4 clip = uProjector.viewProjection * vec4(vWorldPos, 1.0);
    if (clip.w <= 0.0) {
        return vec3(0.0);
    }
    vec3 ndc = clip.xyz / clip.w;
    if (any(lessThan(ndc, vec3(-1.0))) || any(greaterThan(ndc, vec3(1.0)))) {
        return vec3(0.0);
    }
    vec2 uv = ndc.xy * 0.5 + 0.5;
    vec3 toLens = uProjector.position - vWorldPos;
    float d = max(length(toLens), kEpsilon);
```

```
    float lambert = max(dot(N, toLens / d), 0.0);
    float falloff = (uProjector.referenceDistance * uProjector.referenceDistance) /
    (d * d);
    return albedo * texture(uProjector.image, uv).rgb * uProjector.color * lambert *
     falloff;
}
```

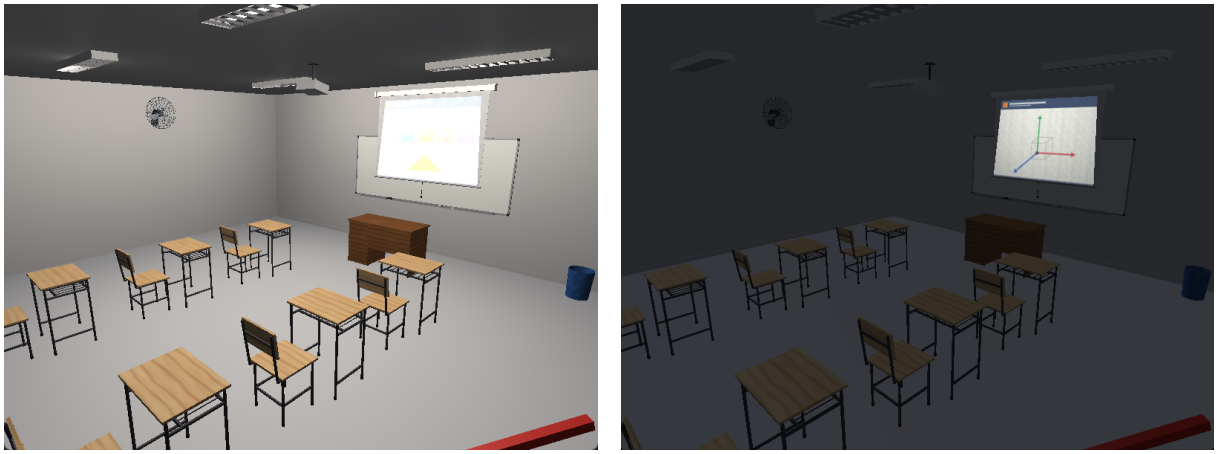
Fonte: Elaborado pelos autores (2026).

A contribuição do projetor é somada às outras luzes:

$$L_{\text{proj}} = \mathbf{a} \odot \text{slide}(u, v) \odot \mathbf{c}_{\text{proj}} \cdot \max(\mathbf{N} \cdot \mathbf{L}, 0) \cdot \left(\frac{d_{\text{ref}}}{d}\right)^2, \tag{5.10}$$

em que $\mathbf{a}$ é a cor difusa da superfície, d a distância até a lente e d_{ref} a distância até o centro da lona, onde o fator vale 1. As partes brancas do slide iluminam; as pretas não projetam nada e a superfície fica com a cor que a sala já lhe dá. A orientação foi conferida pelo bloco laranja no canto superior esquerdo de cada slide: a imagem aparece em pé e sem espelhamento (Figura 4).

Figura 4 – Projeção do slide na tela retrátil



(a) Luzes acesas: pouco contraste (b) Luzes apagadas, segundo slide

Fonte: Elaborada pelos autores (2026).

A principal limitação é a ausência de *shadow mapping*: o shader não sabe se há algo entre a lente e o fragmento. A imagem também chega ao quadro atrás da lona aberta (escondida pela própria lona para quem está na sala), e um objeto à frente do projetor recebe a imagem sem fazer sombra na tela.

6 ILUMINAÇÃO, MATERIAIS E TEXTURAS

6.1 Modelo de Blinn-Phong

A cor é calculada por fragmento no fragment shader, com o modelo de Blinn-Phong (Phong, 1975; Blinn, 1977). Com $\mathbf{N}$ a normal, $\mathbf{L}$ a direção para a luz, $\mathbf{V}$ a direção para a câmera e $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$, todos unitários:

$$\mathbf{cor} = \mathbf{e} + \mathbf{a}_{\text{amb}} \odot \mathbf{a} + \sum_i f_i \, \mathbf{c}_i \odot [\max(\mathbf{N} \cdot \mathbf{L}_i, 0) \, \mathbf{a} + \max(\mathbf{N} \cdot \mathbf{H}_i, 0)^{\alpha} \, \mathbf{k}_s] + L_{\text{proj}}, \tag{6.1}$$

em que $\mathbf{e}$ é a cor emissiva, $\mathbf{a}$ a difusa (albedo), $\mathbf{k}_s$ a especular, α o brilho, $\mathbf{c}_i$ a cor da luz e f_i a atenuação. O termo especular é zerado quando $\mathbf{N} \cdot \mathbf{L} \leq 0$, e os vetores que podem ser nulos são protegidos por um $\varepsilon = 10^{-4}$. O Blinn-Phong foi preferido ao Phong original porque o reflexo enfraquece suavemente em ângulos abertos, sem a borda que $\mathbf{R} \cdot \mathbf{V}$ produz ao ficar negativo, e porque é mais barato (Akenine-Möller *et al.*, 2018).

A sala tem quatro luzes pontuais, uma por luminária, com atenuação

$$f(d) = \frac{1}{k_c + k_l d + k_q d^2}, \qquad k_c = 1, \; k_l = 0{,}14, \; k_q = 0{,}07, \tag{6.2}$$

além de uma luz ambiente constante e de uma luz direcional fraca vinda da esquerda (-X), que representa a claridade do dia e dá volume às faces que as lâmpadas iluminam de lado. A tecla L desliga as luzes pontuais, apaga a emissiva dos tubos e reduz a luz ambiente; a sala escurece, mas não fica preta (Figura 4).

6.2 Matriz das normais

As normais não podem ser transformadas pela mesma matriz dos vértices quando há escala não uniforme, como na lona. Uma normal $\mathbf{n}$ precisa continuar perpendicular a qualquer tangente $\mathbf{t}$ depois da transformação. Se $\mathbf{t}' = M\mathbf{t}$, procura-se G tal que $(G\mathbf{n})^T (M\mathbf{t}) = \mathbf{n}^T (G^T M)\, \mathbf{t} = 0$ para todo $\mathbf{t}$ com $\mathbf{n}^T \mathbf{t} = 0$, o que vale com $G^T M = I$:

$$G = \left(M_{3\times 3}^{-1} \right)^T. \tag{6.3}$$

A matriz é calculada na CPU, uma vez por peça (Código 3.1), e não no vertex shader, uma vez por vértice.

6.3 Materiais e texturas

Cada peça tem um material com cor difusa, textura opcional e sua escala de repetição, cor especular, brilho e cor emissiva. Há predefinições para metal, madeira, plástico e superfícies foscas, e o material é herdado pela árvore: vai na raiz do objeto e só as peças diferentes o

sobrescrevem. As texturas (laminado das carteiras e cadeiras, madeira da mesa do professor, trama da lona, plásticos do projetor e da lixeira e os três slides) são PNGs carregados com stb_image, com *mipmaps*; as coordenadas de textura das primitivas são geradas em metros, então a mesma imagem tem a mesma escala em peças de tamanhos diferentes.

7 CÂMERA E INTERAÇÃO

A câmera livre guarda posição e dois ângulos: guinada (*yaw*, ψ) e arfagem (*pitch*, ϕ). A direção de visão é

$$\mathbf{f} = (\cos\psi\cos\phi,\ \sin\phi,\ \sin\psi\cos\phi), \tag{7.1}$$

e a matriz de visualização é lookAt($\mathbf{p}, \mathbf{p}+\mathbf{f}, \hat{\mathbf{y}}$). A projeção é perspectiva, com 60° de abertura vertical, plano próximo a 0,1 m e distante a 100 m (Vries, 2020). O mouse altera os ângulos a 0,1° por pixel, e a arfagem fica limitada a ±89°: em 90°, $\mathbf{f}$ ficaria paralelo ao vetor "cima" e o `lookAt` não teria como definir a direita. As teclas `W`, `A`, `S` e `D` movem a câmera a 2 m/s no plano da visão e `Espaço` e `Ctrl` a sobem e descem; o deslocamento combinado é normalizado, para a diagonal não ser mais rápida.

O clique esquerdo captura o cursor; com ele livre, não há movimento, mas a inspeção e as interações continuam funcionando. As teclas `1` a `0` e `[/]` selecionam a cena completa ou um dos onze alvos de inspeção. As interações são tratadas em `Classroom::update`, que chama o método do objeto: `L` (luzes), `T` (tela), `V` e `Shift+V` (ventilador), `P` e as setas (projetor). A lista completa está no Apêndice A.

8 RESULTADOS

A sala roda em tempo real em placas de vídeo integradas e também por renderização em software (Mesa llvmpipe). A Figura 5 mostra a tela retrátil aberta pela tecla `T`, antes de o projetor ser ligado; a Figura 4, o slide projetado com as luzes acesas e apagadas. Todas as figuras deste relatório foram capturadas do programa compilado a partir de um clone limpo do repositório.

Figura 5 – Tela retrátil aberta, com o projetor desligado



Fonte: Elaborada pelos autores (2026).

Os requisitos da avaliação foram atendidos: objetos modelados pela equipe, iluminação com materiais e texturas, câmera navegável, inspeção de cada objeto e as três transformações ligadas a interações. O programa compila sem avisos com `-Wall -Wextra -Wpedantic`.

9 COMPILAÇÃO E EXECUÇÃO

O passo a passo completo está no README do repositório. Em resumo, do sistema vêm só o compilador, o CMake, o Git e, no Linux, os headers do X11; as bibliotecas são compiladas a partir de `lib/`. No Windows, com o g++ do MSYS2 (UCRT64) no `Path`:

Código 9.1 – Compilação no Windows (PowerShell)

```
git clone https://github.com/JoseRiquelmeTds/ambiente-sala-de-aula-opengl.git
cd ambiente-sala-de-aula-opengl
cmake --preset windows
cmake --build --preset windows --target run
```

Fonte: Elaborado pelos autores (2026).

No Linux (Ubuntu) e no WSL2:

Código 9.2 – Compilação no Linux

```
sudo apt install -y git build-essential cmake libx11-dev libxrandr-dev \
  libxinerama-dev libxcursor-dev libxi-dev mesa-utils
git clone https://github.com/JoseRiquelmeTds/ambiente-sala-de-aula-opengl.git
cd ambiente-sala-de-aula-opengl
cmake --preset linux
cmake --build --preset linux --target run
```

Fonte: Elaborado pelos autores (2026).

O build limpo da versão final (tag `v1.0.0`) foi testado com um clone novo, seguindo só o README, nos sistemas do Quadro 3.

Quadro 3 – Sistemas testados

Sistema	**Compilador**	**CMake**	**Vídeo**
Windows 11 Pro	g++ 16.2.0 (MSYS2)	4.4.2	Intel Graphics (OpenGL 3.3)
Ubuntu 24.04.5 LTS (contêiner limpo)	g++ 13.3.0	3.28.3	Mesa 25.2.8, llvmpipe (OpenGL 4.5)
Ubuntu 26.04.1 LTS (contêiner limpo)	g++ 15.2.0	4.2.3	Mesa 26.0.8, llvmpipe (OpenGL 4.5)

Fonte: Elaborado pelos autores (2026).

10 DIVISÃO DE TAREFAS

O trabalho foi organizado em tarefas (cards CG-*n*) num quadro Jira, com uma branch e um *pull request* revisado por outro integrante para cada tarefa. O Quadro 4 resume as responsabilidades.

Quadro 4 – Responsabilidades por integrante

Integrante	**Tarefas**
Erasmo da Silva Sá Junior	Build com CMake e bibliotecas embutidas; convenções e arquitetura (CG-53, CG-87); projetor, tela retrátil e lixeira (CG-65 a CG-67); escala da tela (CG-75), rotação do ventilador (CG-76) e projeção do slide (CG-77); texturas da tela, do projetor e da lixeira (CG-78); entrega final (CG-81)
José Riquelme Teixeira da Silva	Malha, primitivas, shader e carregador de texturas (CG-56 a CG-58); carteira, luminária com interruptor e mesa do professor (CG-62 a CG-64); iluminação Blinn-Phong (CG-71), materiais por peça (CG-72), interação da luz (CG-73) e texturas desses objetos (CG-74)
Victor André Lopes Brasileiro	Janela, entrada e loop (CG-54); câmera livre e inspeção (CG-55); ventilador, cadeira e quadro (CG-59 a CG-61); montagem da sala (CG-69); materiais da cadeira, do ventilador e do quadro (CG-74); revisão da navegação (CG-68)

Fonte: Elaborado pelos autores (2026).

11 CONCLUSÃO

O projeto entregou uma sala de aula navegável e interativa usando apenas o pipeline programável do OpenGL 3.3. A organização em grafo de cena foi o ponto central: as três transformações da avaliação se reduzem a trocar a matriz de um único nó articulado (Equação 5.2, Equação 5.3) ou a tratar o projetor como uma câmera (Equação 5.8), sem reconstruir geometria. A escala não uniforme da lona mostrou na prática por que as normais exigem a inversa transposta, e a rotação do ventilador mostrou o efeito da amostragem temporal na escolha das velocidades.

Ficam como limitações e trabalhos futuros a ausência de sombras (*shadow mapping*), que também impediria a imagem do projetor de atravessar a lona, a ausência de colisão da câmera com os objetos e a iluminação apenas local, sem luz rebatida.

REFERÊNCIAS

AKENINE-MÖLLER, Tomas *et al.* **Real-time rendering**. 4. ed. Boca Raton: CRC Press, 2018.

BARRETT, Sean. **stb: single-file public domain libraries for C/C++**. 2026. Disponível em: `https://github.com/nothings/stb`. Acesso em: 1 nov. 2026.

BLINN, James F. Models of light reflection for computer synthesized pictures. *In:* PROCEEDINGS of the 4th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '77). New York: ACM, 1977. p. 192–198.

G-TRUC CREATION. **OpenGL Mathematics (GLM)**. Versão 1.0.3. 2026. Disponível em: `https://github.com/g-truc/glm`. Acesso em: 1 nov. 2026.

GLFW. **GLFW: an OpenGL library**. Versão 3.5.1. 2026. Disponível em: `https://www.glfw.org`. Acesso em: 1 nov. 2026.

HERBERTH, David. **glad: Vulkan/GL/GLES/EGL/GLX/WGL loader-generator**. 2026. Disponível em: `https://github.com/Dav1dde/glad`. Acesso em: 1 nov. 2026.

PHONG, Bui Tuong. Illumination for computer generated pictures. **Communications of the ACM**, v. 18, n. 6, p. 311–317, 1975.

SEGAL, Mark; AKELEY, Kurt. **The OpenGL graphics system: a specification (version 3.3, core profile)**. Beaverton, 2010. Disponível em: `https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf`. Acesso em: 1 nov. 2026.

SEGAL, Mark; KOROBKIN, Carl *et al.* Fast shadows and lighting effects using texture mapping. *In:* PROCEEDINGS of the 19th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '92). New York: ACM, 1992. p. 249–252.

SHIRLEY, Peter; MARSCHNER, Steve. **Fundamentals of computer graphics**. 4. ed. Boca Raton: CRC Press, 2016.

VALE, Helena Cristina Pimentel do; LENZI, Lívia Aparecida Ferreira. **Manual para normalização de trabalhos acadêmicos da UFAL**. 2. ed. Maceió: Sistema de Bibliotecas da UFAL, 2025. Disponível em: `https://www.repositorio.ufal.br/handle/123456789/10627`. Acesso em: 1 nov. 2026.

VRIES, Joey de. **Learn OpenGL: learn modern OpenGL graphics programming in a step-by-step fashion**. [*S. l.*]: Kendall & Welling, 2020. Disponível em: `https://learnopengl.com`. Acesso em: 1 nov. 2026.

APÊNDICE A – LISTA DE COMANDOS

Teclado e mouse com a janela em foco. As teclas são identificadas pela posição no teclado americano, como faz o GLFW; no teclado ABNT2, `[` e `]` são as duas teclas à direita do `P` (as que têm ´ e `[`), e não o `]` perto do Enter.

Quadro 5 – Comandos do programa

Controle	**Ação**
Janela e cursor	
Clique esquerdo	Captura o cursor e ativa a navegação
`Esc`	Libera o cursor; com o cursor já livre, fecha o programa
`Q`	Fecha o programa
Navegação (cursor capturado)	
Mouse	Olha para os lados e para cima/baixo (arfagem limitada a ±89°)
`W` / `S`	Avança / recua na direção da câmera
`A` / `D`	Move para a esquerda / direita
`Espaço` / `Ctrl` esquerdo	Sobe / desce
Inspeção	
`1`	Cena completa
`2` a `9`, `0`	Carteira, objeto articulado, ventilador, luminária, interruptor, mesa do professor, tela retrátil, projetor e lixeira
`[` / `]`	Alvo anterior / próximo, em ciclo pelos onze alvos (inclui cadeira e quadro)
Interações	
`L`	Liga / desliga as luzes do teto (a tecla do interruptor bascula junto)
`T`	Abre / recolhe a tela retrátil (escala)
`V`	Liga / desliga o ventilador (rotação)
`Shift` + `V`	Troca a velocidade do ventilador: 2, 3 ou 4 voltas/s
`P`	Liga / desliga o projetor (projeção)
← / →	Slide anterior / próximo

Fonte: Elaborado pelos autores (2026).